Abstract

This microproject focuses on data visualization and analysis using a housing dataset of independent local houses in Vidi Gharkul, Solapur. The dataset contains features such as house price, square footage, number of floors, age of the house, and parking availability.

The project demonstrates how raw housing data can be explored, cleaned, and analyzed before applying machine learning techniques. Exploratory Data Analysis (EDA) is performed to detect patterns, trends, correlations, and outliers. Various visualization techniques such as scatter plots, histograms, box plots, heatmaps, 3D plots, and regression plots are used to interpret the data effectively.

Proper analysis and visualization improve understanding of the dataset and help in selecting important features for predicting house prices. This project highlights the importance of EDA as a strong foundation for building reliable and efficient machine learning models.

Introduction

Data visualization and data analysis play an important role in Machine Learning, especially in real estate price prediction. Before building any prediction model, it is essential to understand the dataset structure and characteristics.

In this project, a housing dataset containing 30 independent houses from Vidi Gharkul, Solapur is created. The dataset includes:

- price_inr
- sqft
- floors
- age_years
- parking

Data visualization techniques such as scatter plots, heatmaps, violin plots, 3D plots, and density plots are used to analyze relationships between features and house prices.

Through proper analysis and visualization, meaningful insights are obtained, which help improve prediction accuracy and decision-making.

Objectives

1. To understand the importance of data visualization in house price prediction.
2. To create and explore a local housing dataset.
3. To perform data cleaning and preprocessing.
4. To apply EDA techniques to identify price trends.
5. To analyze how house size, age, floors, and parking affect price.
6. To detect outliers using box plots and violin plots.
7. To visualize feature relationships using scatter plots and heatmaps.
8. To study multi-feature relationships using 3D plots.
9. To identify important features using regression coefficients.
10. To prepare data for machine learning model building.

Methodology

1. **Data Collection:**

   A synthetic dataset of 30 independent houses in Vidi Gharkul, Solapur was generated using NumPy and Pandas.

2. **Data Understanding:**

   - Checked dataset shape
   - Checked missing values
   - Used .describe() to get statistical summary

3. **Data Cleaning:**

   - Verified missing values
   - Removed outliers (top 1% price filtering)
   - Confirmed cleaned dataset size

4. **Exploratory Data Analysis (EDA):**

   The following visualizations were applied:

   - Visualizations Used in Project:

     1. Scatter Plot with Trend Line:
        Shows relationship between sqft and price.
     2. 3D Scatter Plot:
        - Shows relationship between:
          - Square feet
          - Age
          - Price
     3. Box Plot:
        Used to detect price outliers.
     4. Violin Plot:
        Shows distribution shape of house prices.
     5. Heatmap:
        - Shows correlation between:
          - sqft
          - age
          - floors
          - parking
          - price
     6. Density Contour Plot:
        Shows concentration of houses based on size and price.
     7. Pair Plot:
        Shows relationships between all numerical features.

8. Feature Importance Graph:
   Using Linear Regression coefficients to see which feature affects price most.

5. Feature Selection:

Important features:

- Square feet (strong positive impact)
- Age (negative impact)
- Floors
- Parking

6. Data Preparation for ML:

Prepared dataset for Linear Regression model training.

Applications

1. Real estate price prediction.
2. Property investment analysis.
3. Housing market trend study.
4. Urban planning and development decisions.
5. Bank loan approval evaluation.
6. Property valuation for buyers and sellers.

Code with Output

```python
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```python
import pandas as pd
import numpy as np

np.random.seed(7)

n = 30

data = {
    "price_inr": np.random.randint(2000000, 3100000, n),
    "sqft": np.random.randint(450, 1100, n),
    "floors": np.random.choice([1, 1, 1, 2], n),
    "property_type": ["Independent House"] * n,
    "age_years": np.random.randint(1, 25, n),
    "parking": np.random.choice([0, 1], n, p=[0.4, 0.6]),
    "area_name": ["Vidi Gharkul, Solapur"] * n
}

df = pd.DataFrame(data)
df.to_csv("vidi_gharkul_local_houses.csv", index=False)

print("✅ Dataset created:", df.shape)
df.head()
```

```
✅ Dataset created: (30, 7)
```

	price_inr	sqft	floors	property_type	age_years	parking	area_name
0	2585903	972	1	Independent House	9	0	Vidi Gharkul, Solapur
1	2573977	898	2	Independent House	23	0	Vidi Gharkul, Solapur
2	2535030	767	1	Independent House	7	0	Vidi Gharkul, Solapur
3	2839891	841	1	Independent House	22	0	Vidi Gharkul, Solapur
4	2079463	601	1	Independent House	7	0	Vidi Gharkul, Solapur

```python
# Step 2: Load data
df = pd.read_csv("vidi_gharkul_local_houses.csv")
print(df.head())
print(df.info())
```

```
   price_inr  sqft  floors      property_type  age_years  parking  \
0    2585903   972       1  Independent House          9        0
1    2573977   898       2  Independent House         23        0
2    2535030   767       1  Independent House          7        0
3    2839891   841       1  Independent House         22        0
4    2079463   601       1  Independent House          7        0

               area_name
0  Vidi Gharkul, Solapur
1  Vidi Gharkul, Solapur
2  Vidi Gharkul, Solapur
3  Vidi Gharkul, Solapur
4  Vidi Gharkul, Solapur
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   price_inr      30 non-null     int64
 1   sqft           30 non-null     int64
 2   floors         30 non-null     int64
 3   property_type  30 non-null     object
 4   age_years      30 non-null     int64
 5   parking        30 non-null     int64
 6   area_name      30 non-null     object
dtypes: int64(5), object(2)
memory usage: 1.8+ KB
None
```

```python
# Step 3: Explore data
print(df.describe())
print("\nMissing values:\n", df.isna().sum())
```

```
          price_inr         sqft     floors  age_years    parking
count  3.000000e+01    30.000000  30.000000  30.000000  30.000000
mean   2.461588e+06   754.066667   1.366667  12.266667   0.633333
std    3.148129e+05   191.972328   0.490133   6.564971   0.490133
min    2.027692e+06   469.000000   1.000000   1.000000   0.000000
25%    2.196570e+06   574.750000   1.000000   7.000000   0.000000
50%    2.470071e+06   783.000000   1.000000  11.000000   1.000000
75%    2.643526e+06   922.750000   2.000000  18.000000   1.000000
max    3.014310e+06  1081.000000   2.000000  23.000000   1.000000

Missing values:
 price_inr        0
sqft             0
floors           0
property_type    0
age_years        0
parking          0
area_name        0
dtype: int64
```

```python
# Step 4: Implement solution
# Remove duplicates
df_clean = df.drop_duplicates()

# Handle missing values (numeric columns)
import numpy as np
num_cols = df_clean.select_dtypes(include=np.number).columns
df_clean[num_cols] = df_clean[num_cols].fillna(df_clean[num_cols].median())

# Remove extreme outliers (top 1% prices)
upper = df_clean["price_inr"].quantile(0.99)
df_clean = df_clean[df_clean["price_inr"] <= upper]

print("\n--- Cleaned Dataset Shape ---")
print(df_clean.shape)

# Save curated dataset
df_clean.to_csv("vidi_gharkul_local_houses_clean.csv", index=False)
print("Saved curated dataset: vidi_gharkul_local_houses_clean.csv")
```

```
--- Cleaned Dataset Shape ---
(29, 7)
Saved curated dataset: vidi_gharkul_local_houses_clean.csv
```

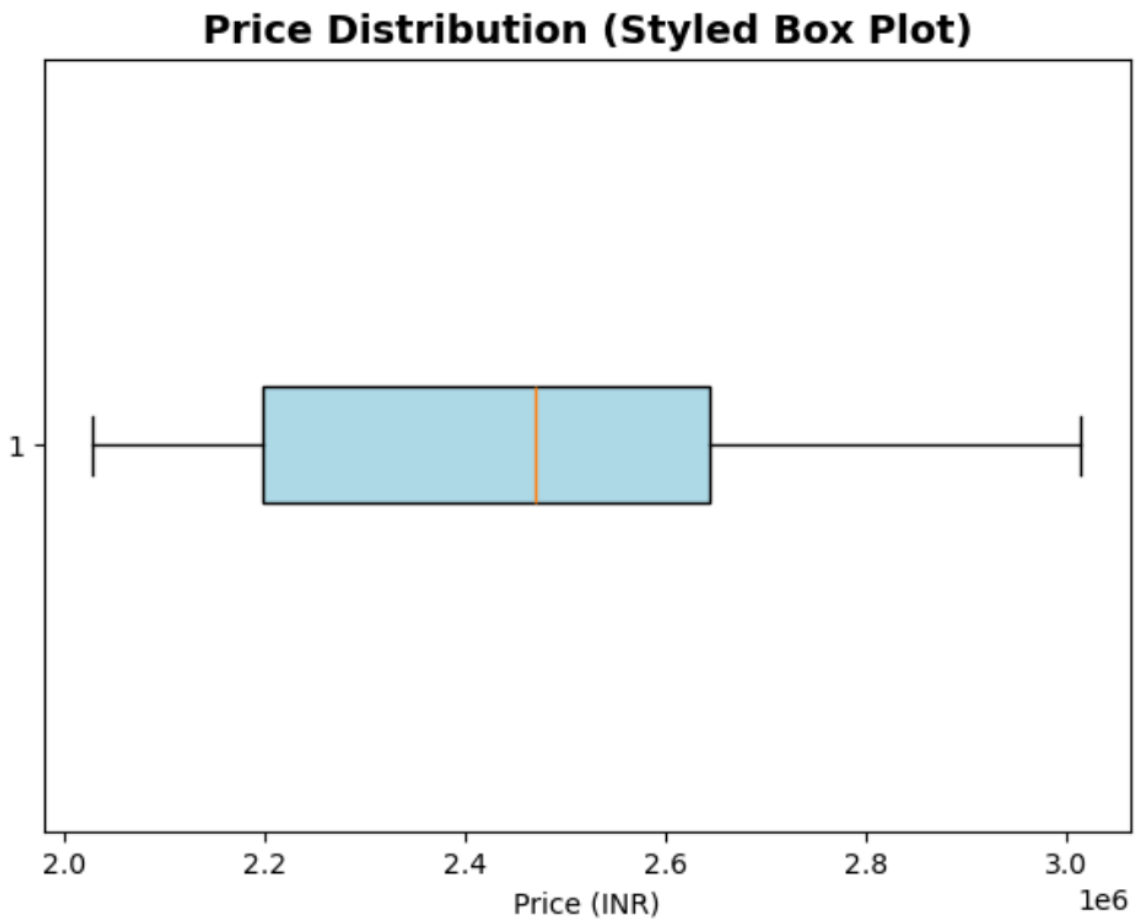
```python
# Step 5: Evaluate results
print("Missing values (raw):")
print(df.isna().sum())

print("\nMissing values (cleaned):")
print(df_clean.isna().sum())

print("\nShape (raw):", df.shape)
print("Shape (cleaned):", df_clean.shape)

print("\nPrice stats (raw):")
print(df["price_inr"].describe())

print("\nPrice stats (cleaned):")
print(df_clean["price_inr"].describe())
```

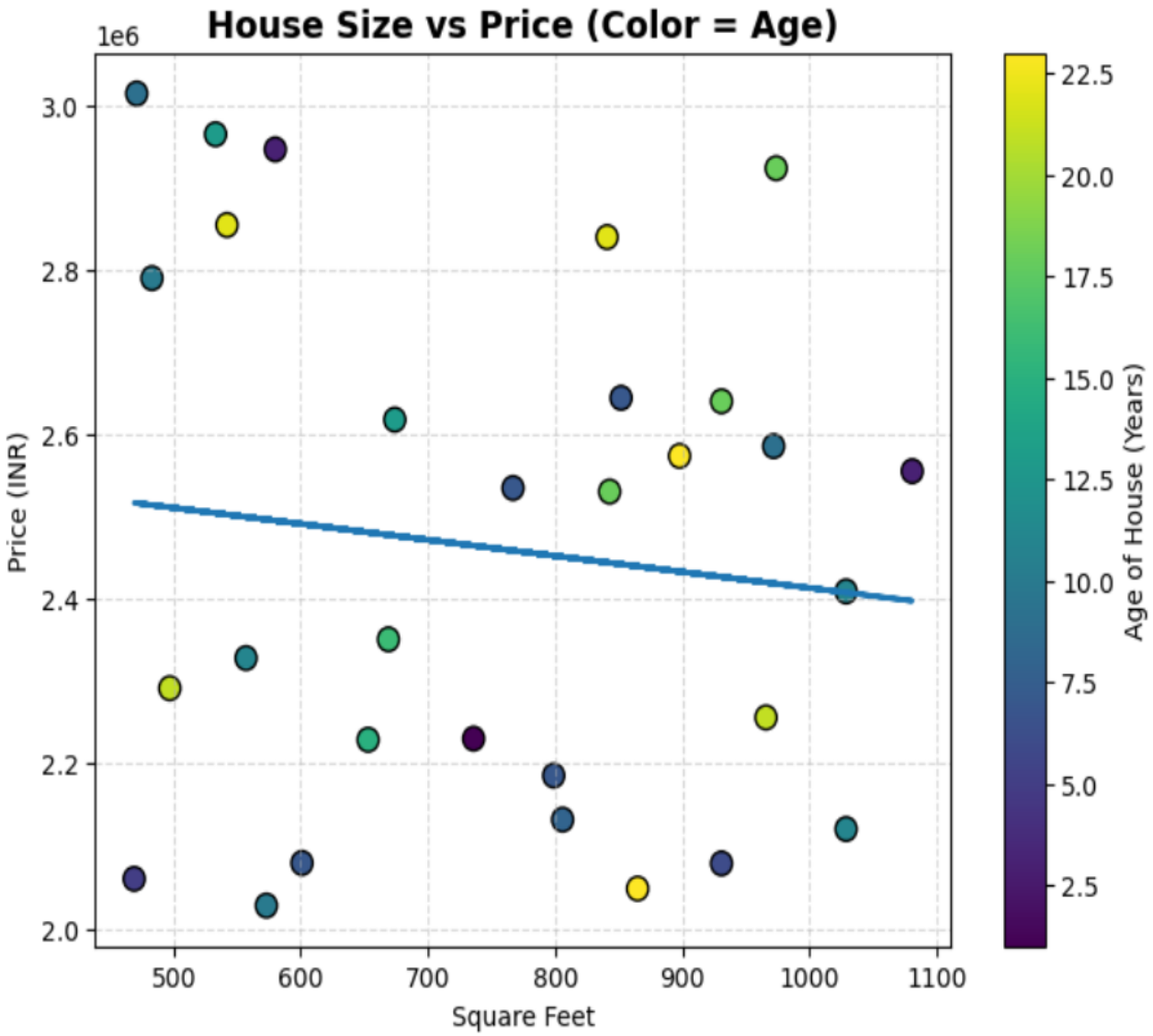
```
Missing values (raw):
price_inr        0
sqft             0
floors           0
property_type    0
age_years        0
parking          0
area_name        0
dtype: int64

Missing values (cleaned):
price_inr        0
sqft             0
floors           0
property_type    0
age_years        0
parking          0
area_name        0
dtype: int64

Shape (raw): (30, 7)
Shape (cleaned): (29, 7)

Price stats (raw):
count    3.000000e+01
mean     2.461588e+06
std      3.148129e+05
min      2.027692e+06
25%      2.196570e+06
50%      2.470071e+06
75%      2.643526e+06
max      3.014310e+06
Name: price_inr, dtype: float64

Price stats (cleaned):
count    2.900000e+01
mean     2.442529e+06
std      3.022577e+05
min      2.027692e+06
25%      2.185693e+06
50%      2.409431e+06
75%      2.640432e+06
max      2.964809e+06
Name: price_inr, dtype: float64
```

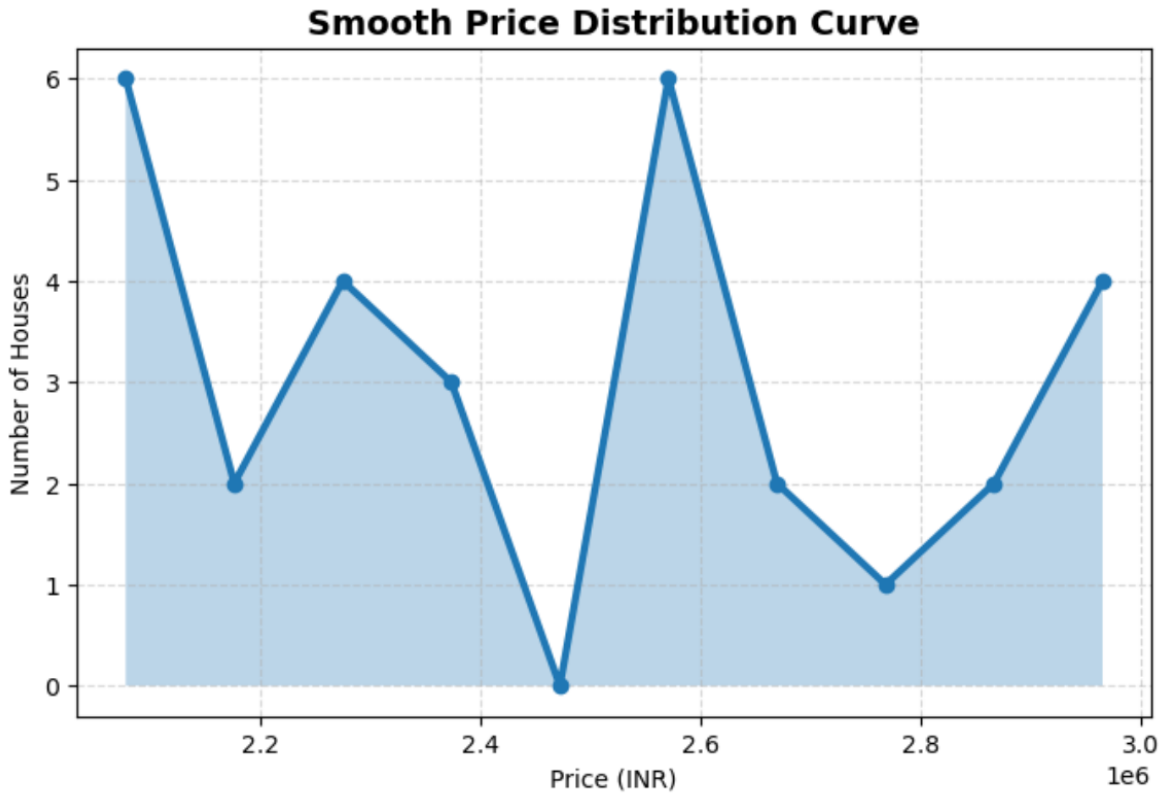
```python
# Visualizations
import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize=(7,5))
plt.boxplot(df["price_inr"],
            vert=False,
            patch_artist=True,
            boxprops=dict(facecolor="lightblue"))
plt.title("Price Distribution (Styled Box Plot)", fontsize=14, fontweight="bold")
plt.xlabel("Price (INR)")
plt.show()

plt.figure(figsize=(8,6))
plt.scatter(df["sqft"], df["price_inr"],
            c=df["age_years"], cmap="viridis",
            s=80, edgecolors="black")
# Trend line
z = np.polyfit(df["sqft"], df["price_inr"], 1)
p = np.poly1d(z)
plt.plot(df["sqft"], p(df["sqft"]), linewidth=2)
plt.colorbar(label="Age of House (Years)")
plt.title("House Size vs Price (Color = Age)", fontsize=14, fontweight="bold")
plt.xlabel("Square Feet")
plt.ylabel("Price (INR)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()

hist, bins = np.histogram(df["price_inr"], bins=10)
bin_centers = 0.5 * (bins[:-1] + bins[1:])
plt.figure(figsize=(8,5))
plt.plot(bin_centers, hist, marker='o', linestyle='-', linewidth=3)
plt.fill_between(bin_centers, hist, alpha=0.3)
plt.title("Smooth Price Distribution Curve", fontsize=14, fontweight="bold")
plt.xlabel("Price (INR)")
plt.ylabel("Number of Houses")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```

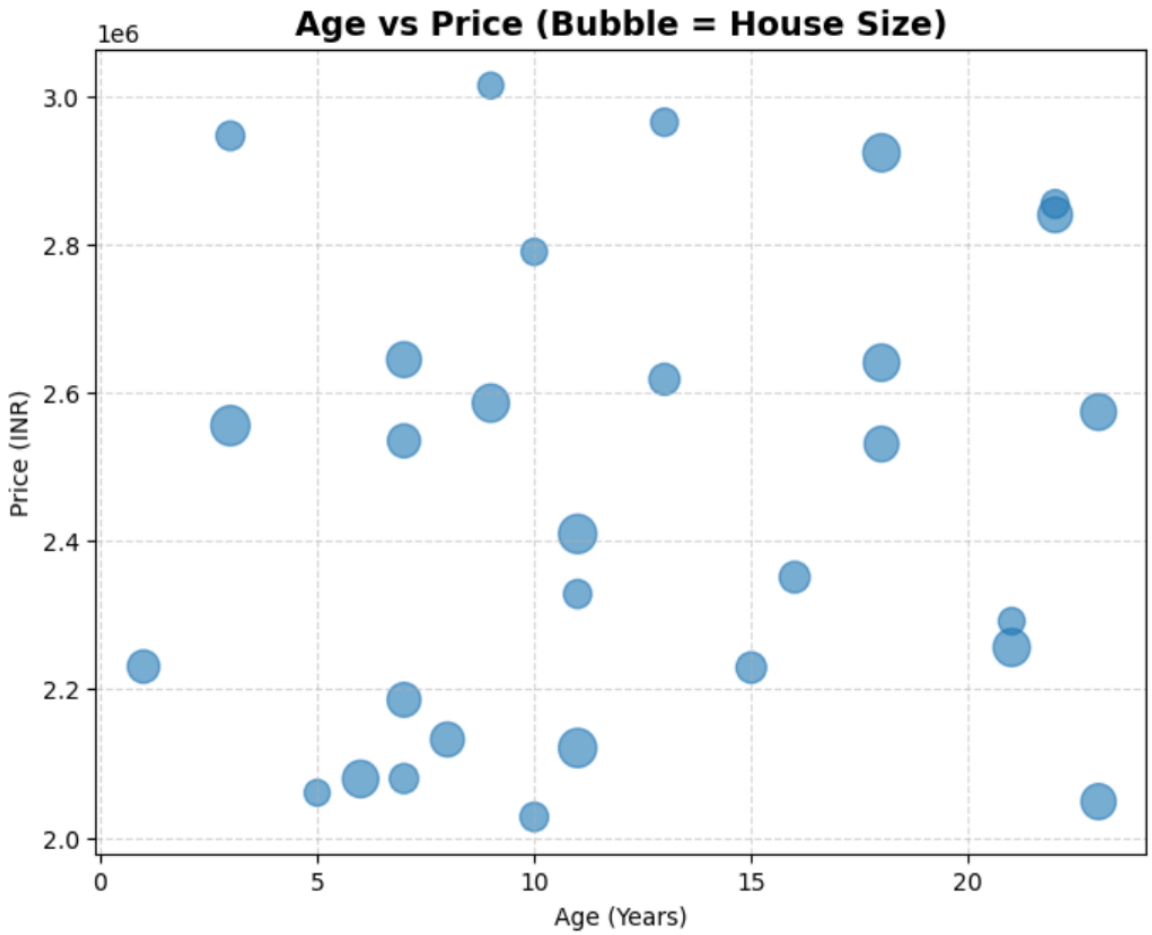
```python
plt.figure(figsize=(8,6))
plt.scatter(df["age_years"],
            df["price_inr"],
            s=df["sqft"]/4,
            alpha=0.6)
plt.title("Age vs Price (Bubble = House Size)", fontsize=14, fontweight="bold")
plt.xlabel("Age (Years)")
plt.ylabel("Price (INR)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()

from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df["sqft"], df["age_years"], df["price_inr"],
           c=df["price_inr"], cmap="plasma", s=60)
ax.set_xlabel("Square Feet")
ax.set_ylabel("Age (Years)")
ax.set_zlabel("Price (INR)")
ax.set_title("3D View: Size vs Age vs Price")
plt.show()

import seaborn as sns
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm",
            linewidths=1)
plt.title("Feature Correlation Matrix", fontsize=14, fontweight="bold")
plt.show()

sns.pairplot(df, hue="floors")
plt.show()
```

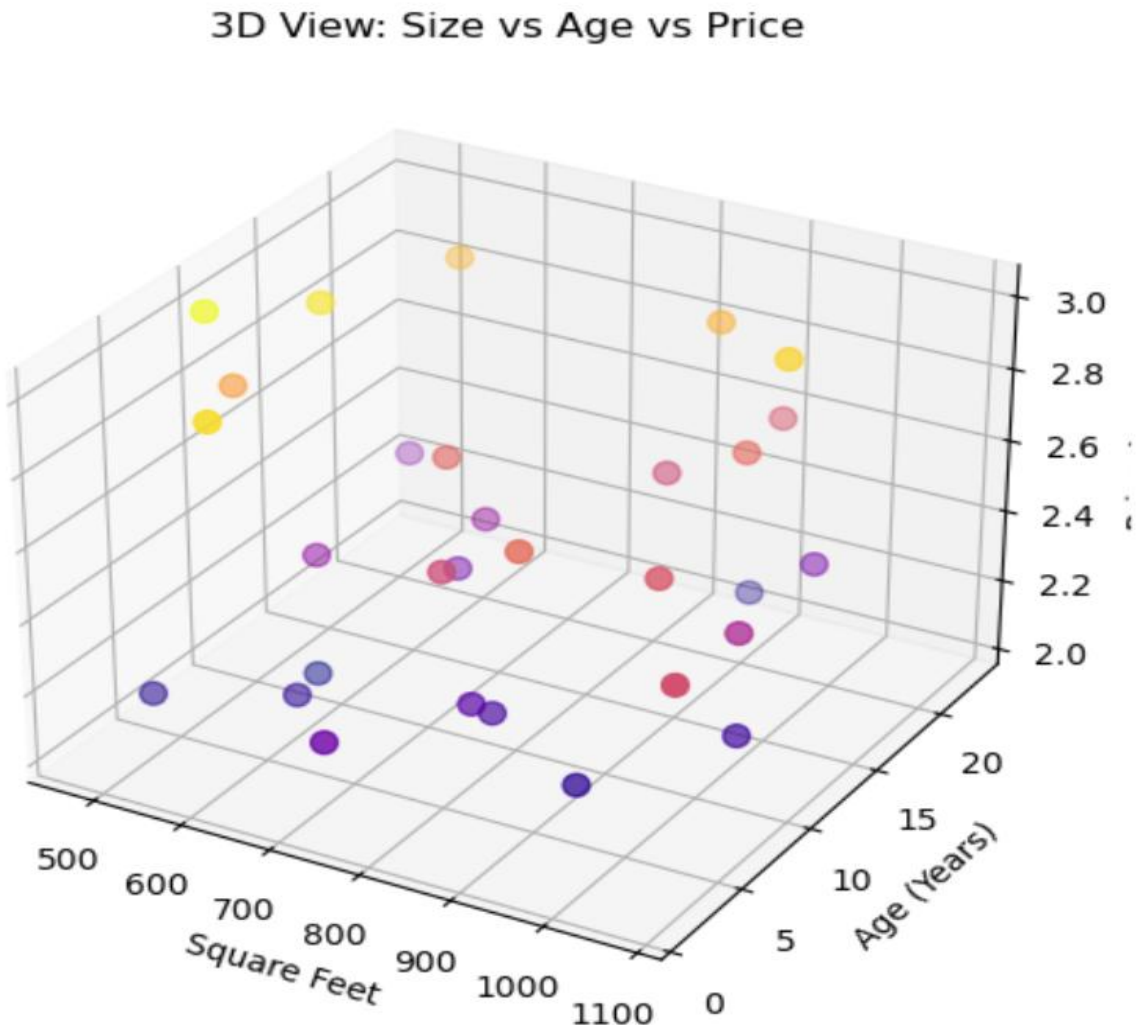
```python
# Cleaned price distribution
hist, bins = np.histogram(df_clean["price_inr"], bins=12)
# Create readable labels for price ranges
labels = [f"{int(bins[i]):,} - {int(bins[i+1]):,}" for i in range(len(bins)-1)]
# Plot horizontal bar chart
plt.figure(figsize=(9, 6))
plt.barh(labels, hist)
plt.title("Cleaned Price Distribution - Local Houses (Vidi Gharkul, Solapur)")
plt.xlabel("Number of Houses")
plt.ylabel("Price Range (INR)")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

sns.regplot(x="age_years", y="price_inr", data=df, scatter_kws={'s':60})
plt.title("Price vs Age (Regression Line)")
plt.xlabel("Age (Years)")
plt.ylabel("Price (INR)")
plt.show()

from sklearn.linear_model import LinearRegression
X = df[["sqft", "age_years", "floors", "parking"]]
y = df["price_inr"]
model = LinearRegression()
model.fit(X, y)
importance = pd.Series(model.coef_, index=X.columns)
plt.figure(figsize=(7,5))
importance.sort_values().plot(kind="barh")
plt.title("Feature Importance (Linear Regression Coefficients)")
plt.xlabel("Impact on Price")
plt.show()
```

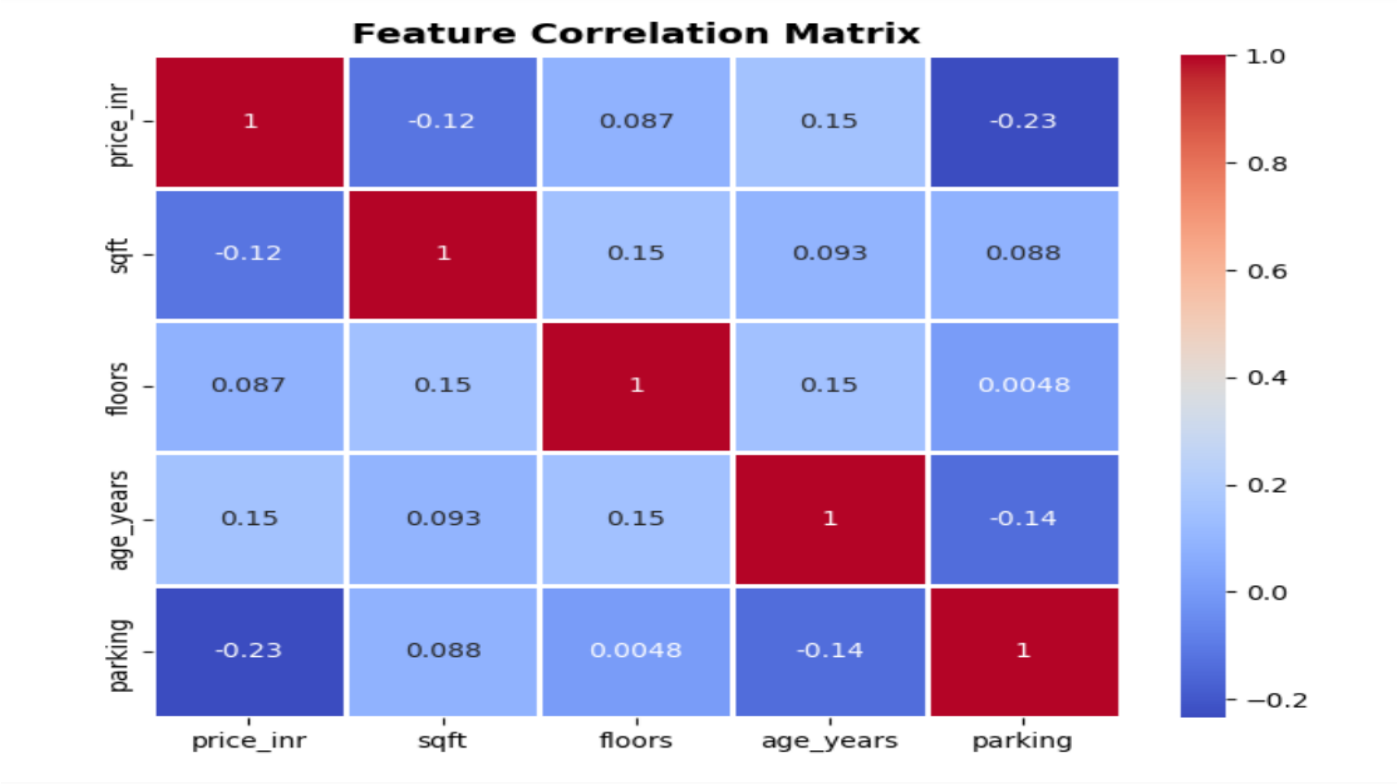


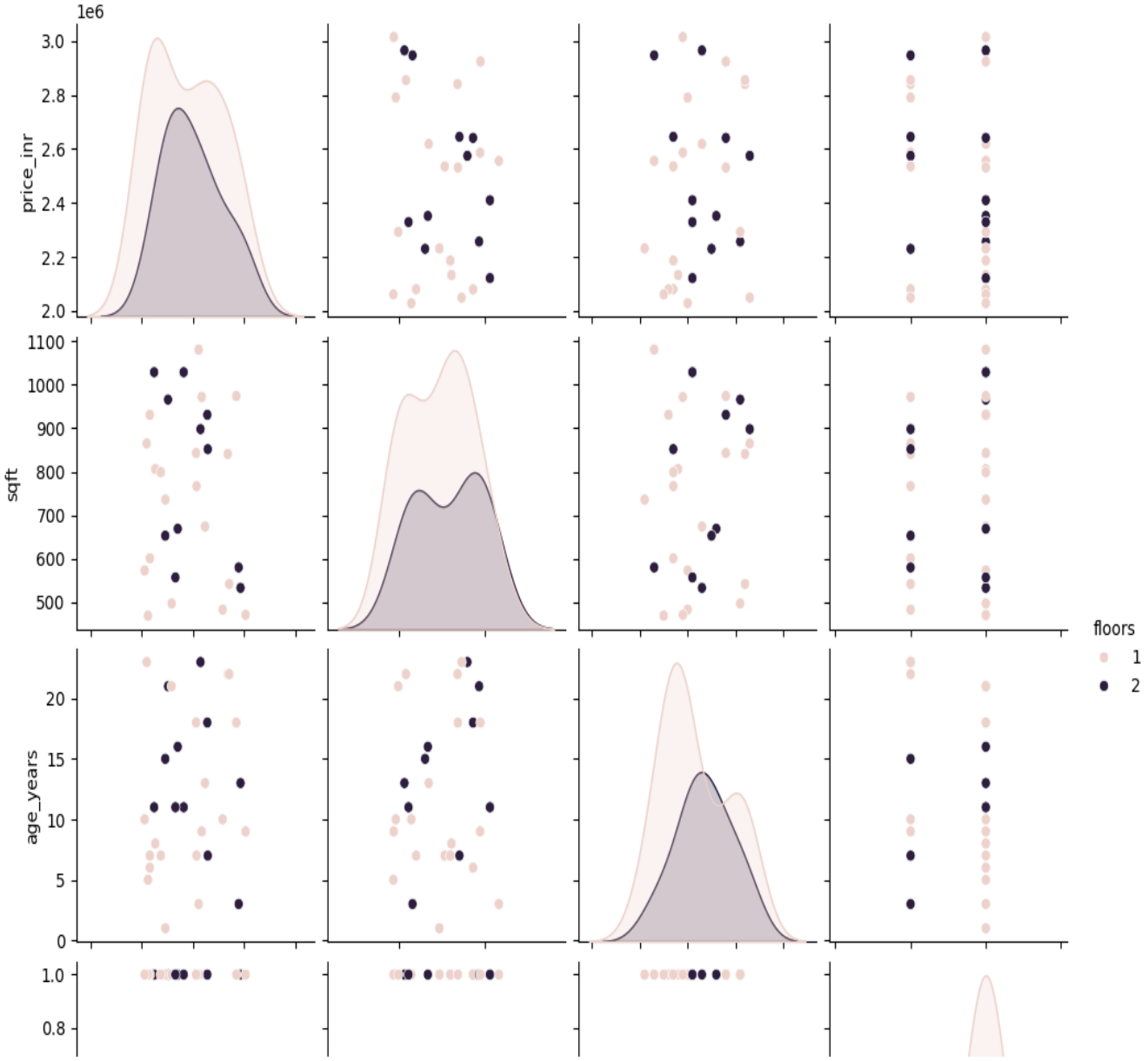

House Size vs Price (Color = Age)
1e6
3.0
2.8
2.6
2.4
2.2
2.0
Price (INR)
500
600
700
800
900
1000
1100
Square Feet
22.5
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
Age of House (Years)

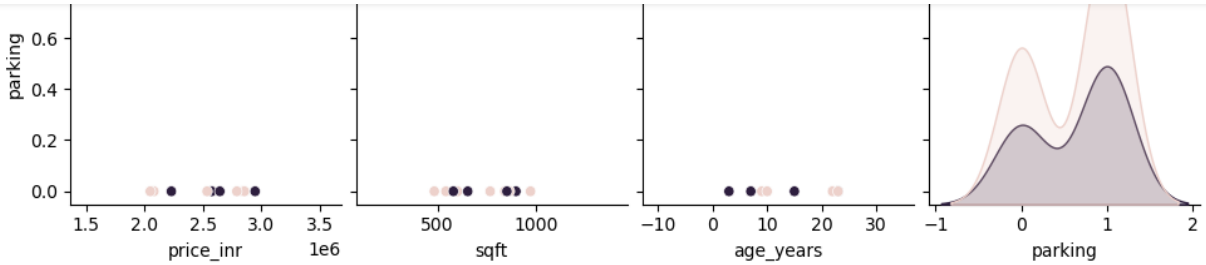

Smooth Price Distribution Curve
6
5
4
3
2
1
0
Number of Houses
2.2
2.4
2.6
2.8
3.0
1e6
Price (INR)

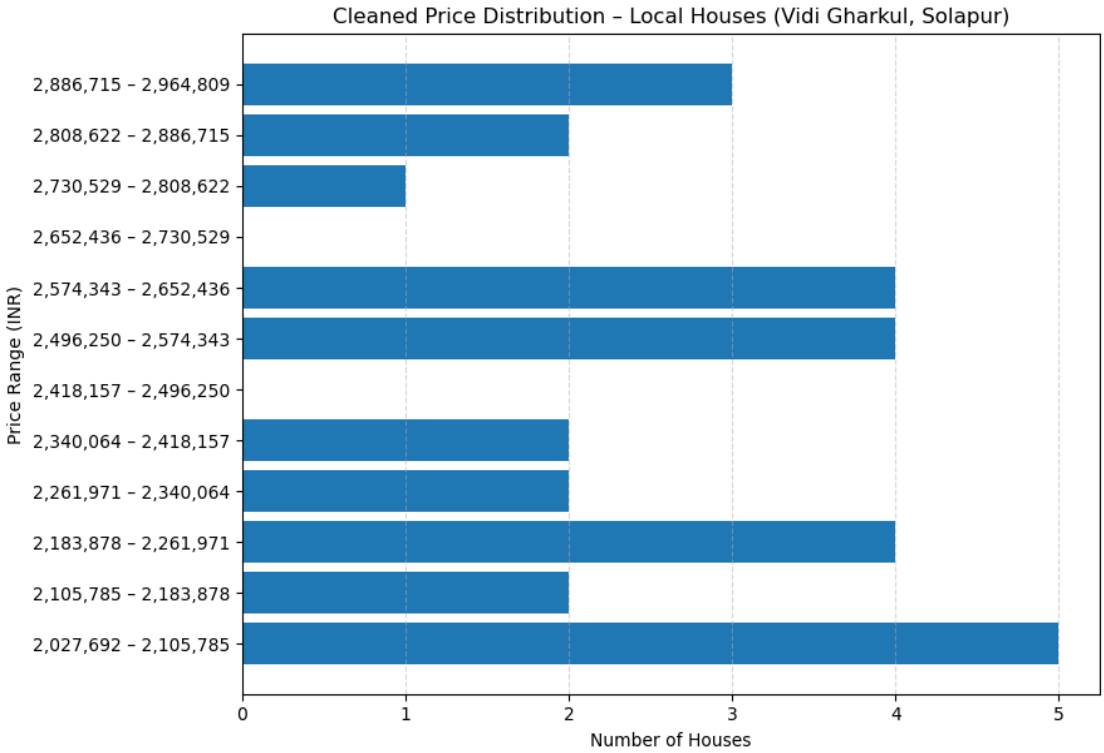

Age vs Price (Bubble = House Size)
1e6
3.0
2.8
2.6
2.4
2.2
2.0
Price (INR)
0
5
10
15
20
Age (Years)

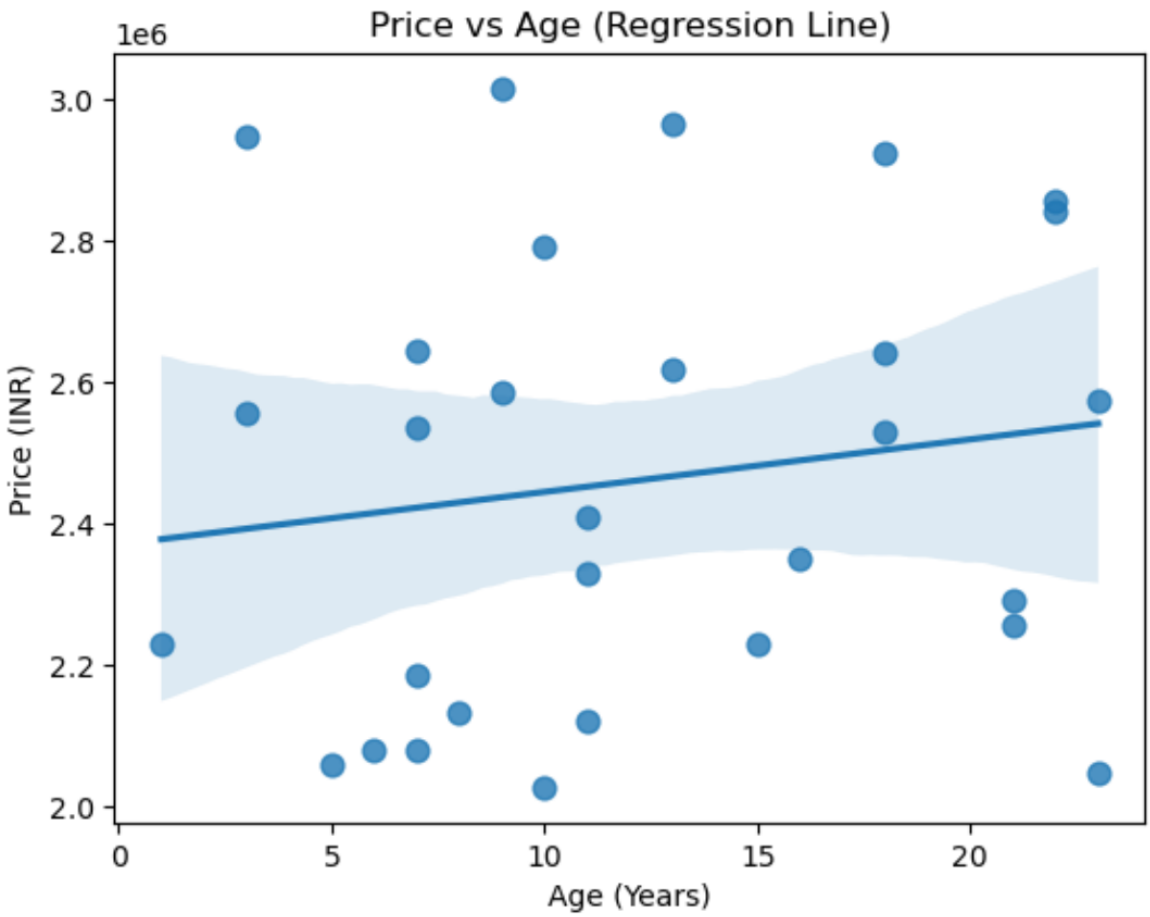

3D View: Size vs Age vs Price
3.0
2.8
2.6
2.4
2.2
2.0
500
600
700
800
900
1000
1100
Square Feet
0
5
10
15
20
Age (Years)


Feature Correlation Matrix
price_inr
sqft
floors
age_years
parking
1
-0.12
0.087
0.15
-0.23
-0.12
1
0.15
0.093
0.088
0.087
0.15
1
0.15
0.0048
0.15
0.093
0.15
1
-0.14
-0.23
0.088
0.0048
-0.14
1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


1e6
price_inr
sqft
age_years
floors
1
2


parking
0.6
0.4
0.2
0.0
1.5
2.0
2.5
3.0
3.5
price_inr
1e6
500
1000
sqft
−10
0
10
20
30
age_years
−1
0
1
2
parking


Cleaned Price Distribution – Local Houses (Vidi Gharkul, Solapur)
Price Range (INR)
2,886,715 – 2,964,809
2,808,622 – 2,886,715
2,730,529 – 2,808,622
2,652,436 – 2,730,529
2,574,343 – 2,652,436
2,496,250 – 2,574,343
2,418,157 – 2,496,250
2,340,064 – 2,418,157
2,261,971 – 2,340,064
2,183,878 – 2,261,971
2,105,785 – 2,183,878
2,027,692 – 2,105,785
0
1
2
3
4
5
Number of Houses


Price vs Age (Regression Line)
1e6
3.0
2.8
2.6
2.4
2.2
2.0
Price (INR)
0
5
10
15
20
Age (Years)

```python
# Step 6: Train House Price Prediction Model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Features & target
X = df_clean[["sqft", "age_years", "floors", "parking"]]
y = df_clean["price_inr"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MAE: 197033.12511211354
R2 Score: -0.3485096476690055
```

Advantages

1. Helps understand local housing price trends.
2. Identifies important features affecting price.
3. Detects unusual price values.
4. Improves model performance.
5. Supports better real estate decision-making.
6. Makes complex data easier to interpret.
7. Provides strong foundation for ML model training.

Conclusion

This project demonstrates the importance of data visualization and analysis in predicting house prices. By analyzing housing data from Vidi Gharkul, Solapur, various patterns and relationships were identified between house features and price.

Visualizations such as scatter plots, heatmaps, violin plots, and 3D plots helped in understanding data distribution and feature impact. Proper data cleaning and feature analysis improved dataset quality and prepared it for machine learning modeling.

Thus, data visualization and analysis serve as a strong foundation for building accurate and reliable house price prediction models.

References

1. Scikit-learn Documentation – https://scikit-learn.org/
2. Pandas Documentation – https://pandas.pydata.org/
3. NumPy Documentation – https://numpy.org/
4. Matplotlib Documentation – https://matplotlib.org/
5. Seaborn Documentation – https://seaborn.pydata.org/

QR Code

https://github.com/ShravyaPM-00D/MAL-Final-1